République Tunisienne
Ministère de l'Enseignement Supérieur
et de la Recherche Scientifique

Université de Sfax

École Nationale d'Ingénieurs
de Sfax

Département :
Génie Informatique
et Mathématiques Appliquées

Rapport de stage

présenté à

L'École Nationale d'Ingénieurs de Sfax

filière

Génie Informatique et Mathématiques Appliquées

Réalisé au sein de :

Decade Sfax

par

Najmeddine Maatoug

Projet IA e-commerce : enrichissement et optimisation des données produit

Sous l'encadrement de :

Mme [Nom Encadrante ENIS]	Encadrante académique
M. [Nom Encadrant Decade]	Encadrant industriel

REMERCIEMENTS

Je tiens tout d'abord à remercier **Madame [Nom Encadrante ENIS]**, mon encadrante académique à l'ENIS, pour son accompagnement, la qualité de ses orientations scientifiques et sa disponibilité tout au long de ce travail.

J'exprime également ma profonde gratitude à **Monsieur [Nom Encadrant Decade]**, mon encadrant industriel chez Decade Sfax, pour sa confiance, ses conseils techniques et son suivi régulier durant la réalisation du projet.

Mes remerciements s'adressent aussi à **toute l'équipe de Decade Sfax** pour son accueil, son professionnalisme et l'environnement de travail stimulant qu'elle m'a offert. Les échanges avec les membres de l'équipe ont été déterminants pour la réussite de ce stage.

Je remercie vivement **le corps enseignant de l'ENIS** pour la qualité de la formation reçue, qui m'a permis d'aborder ce projet avec des bases solides en ingénierie logicielle et en intelligence artificielle.

Enfin, j'adresse mes plus sincères remerciements à **ma famille et à mes proches** pour leur soutien moral, leur patience et leurs encouragements constants.

TABLE DES MATIÈRES

INTRODUCTION GÉNÉRALE **2**

1 Cadre général du projet **4**

1.1 Introduction 5

1.2 Présentation de l'organisme d'accueil 5

1.2.1 DECADE Tunisie et contexte d'activité 5

1.2.2 Cadre académique du stage 5

1.3 Problématique 5

1.4 Objectifs du projet 6

1.4.1 Objectif principal 6

1.4.2 Objectifs spécifiques 6

1.4.3 Indicateurs cibles 6

1.5 Acteurs et interactions principales 7

1.6 Périmètre et contraintes 7

1.6.1 Périmètre fonctionnel 7

1.6.2 Contraintes de réalisation 7

1.7 Démarche méthodologique et planification 8

1.8 Conclusion 8

2 Analyse des besoins et étude de l'existant **9**

2.1 Introduction 10

2.2 Analyse des besoins 10

2.2.1 Besoins fonctionnels 10

2.2.2 Besoins non fonctionnels 11

2.2.3 Acteurs et cas d'usage 11

2.2.4 Priorisation des user stories 12

2.3 Étude de l'existant 13

2.3.1 Objectif et protocole des tests 13

2.3.2 Axe 1 – Solutions gratuites locales (Ollama) 14

2.3.2.1 Résultats observés 14
2.3.2.2 Variante Colab GPU gratuit 14
2.3.3 Axe 2 – APIs gratuites (OpenRouter, Groq, AI Studio Free) 16
2.3.3.1 OpenRouter (free tier) 16
2.3.3.2 Groq (free tier) 16
2.3.3.3 Google AI Studio (gratuit) 17
2.3.4 Axe 3 – APIs payantes (candidats de production) 17
2.3.5 Synthèse de décision 18
2.4 Conclusion 18

INTRODUCTION GÉNÉRALE

La transformation numérique du commerce électronique a profondément modifié la manière dont les entreprises conçoivent, valorisent et diffusent leurs catalogues produits. Dans cet écosystème, la donnée produit n'est plus un simple support descriptif : elle devient un levier stratégique qui conditionne la qualité de la recherche, la pertinence des recommandations et, au final, la conversion commerciale.

Dans le contexte de **DECADE Sfax**, cette problématique prend une dimension opérationnelle forte. Les fiches produit, gérées dans un environnement **SAP Commerce Cloud (Hybris)** et exploitées par **Apache Solr**, présentent souvent des attributs incomplets ou hétérogènes. Cette situation dégrade l'expérience utilisateur : filtres peu fiables, recherche imprécise, difficultés de navigation et perte de visibilité de certains articles.

Le présent projet de fin d'études répond à ce besoin en proposant une démarche d'**enrichissement automatique des données produit par IA multimodale**. L'idée directrice consiste à mobiliser un *Large Language Model* capable d'analyser des informations textuelles et visuelles afin de suggérer des attributs structurés (couleur, matière, catégorie, motif, saisonnalité, description normalisée), puis d'intégrer ces enrichissements dans une chaîne maîtrisée de validation et d'exploitation.

Au-delà de l'aspect algorithmique, l'enjeu est d'ordre **industriel** : garantir la fiabilité des données générées, préserver la cohérence avec le modèle Hybris existant, et améliorer l'indexation Solr sans perturber les processus métier. Le projet s'inscrit ainsi dans une logique de compromis entre innovation IA, gouvernance de la donnée et contraintes de production.

Ce rapport est organisé de manière progressive. Le **chapitre 1** présente le cadre général du projet : organisme d'accueil, problématique, objectifs et périmètre. Les chapitres suivants détaillent l'analyse, la conception, la réalisation, puis la validation de la solution proposée.

Cette introduction pose donc le fil conducteur du mémoire : *transformer une problématique de qualité de données en une opportunité d'amélioration mesurable de la performance e-commerce, grâce à une IA multimodale intégrée de façon pragmatique dans l'existant*.

Chapitre

1 Cadre général du projet

1.1 Introduction

Ce chapitre introduit le cadre institutionnel, organisationnel et technique du projet. Il précise le besoin métier, les objectifs poursuivis, les acteurs impliqués ainsi que le périmètre de réalisation retenu.

1.2 Présentation de l'organisme d'accueil

1.2.1 DECADE Tunisie et contexte d'activité

DECADE Tunisie est une filiale spécialisée dans l'ingénierie e-commerce. Le stage s'est déroulé au sein de l'entité de Sfax, dans un contexte de projets orientés qualité de données, performance de recherche et optimisation des parcours utilisateurs.

L'environnement technique de référence mobilise notamment **SAP Commerce Cloud (Hybris)** pour la gestion du catalogue, **Apache Solr** pour la recherche et les facettes, ainsi qu'une architecture orientée **API REST** pour l'interopérabilité des services.

1.2.2 Cadre académique du stage

Ce travail s'inscrit dans le cadre du projet de fin d'études de la filière Génie Informatique et Mathématiques Appliquées de l'ENIS, sur la période de février à juillet 2026. Il vise à produire une contribution concrète, exploitable et argumentée scientifiquement.

1.3 Problématique

Dans un catalogue e-commerce volumineux, la qualité des fiches produit conditionne directement la pertinence de la recherche. Or, l'enrichissement manuel des attributs présente plusieurs limites :

— forte charge opérationnelle pour les équipes métier ;

— hétérogénéité de saisie (valeurs redondantes, variantes lexicales, omissions) ;

— retard d'indexation de données utiles pour Solr ;

— impact négatif sur les filtres dynamiques et le ranking.

La problématique centrale peut être formulée ainsi :
Comment automatiser l'enrichissement des attributs produit à partir de données multimodales, tout en assurant la qualité, la traçabilité et l'intégration native dans l'écosystème Hybris/Solr ?

1.4 Objectifs du projet

1.4.1 Objectif principal

Concevoir une solution d'enrichissement automatique des données produit par IA multimodale, intégrée à SAP Commerce Cloud et exploitable dans Apache Solr pour améliorer la recherche e-commerce.

1.4.2 Objectifs spécifiques

— Extraire automatiquement des attributs structurés (couleur, matière, catégorie, motif, saisonnalité, description) ;

— Mettre en place un espace de validation humaine (validation, correction, rejet) pour maîtriser la qualité ;

— Injecter les attributs validés dans Hybris avec une logique de mapping et de gestion de statuts ;

— Alimenter Solr pour améliorer facettes, filtres et pertinence des résultats ;

— Exposer les données enrichies via des services API pour faciliter la réutilisation.

1.4.3 Indicateurs cibles

Afin d'évaluer l'apport de la solution, les indicateurs suivants sont retenus :

— taux de complétion des attributs avant/après enrichissement ;

— part des suggestions validées manuellement ;

— score de confiance moyen par attribut ;

— impact sur la qualité de recherche (facettes disponibles, précision perçue).

1.5 Acteurs et interactions principales

Le système met en jeu des acteurs humains et techniques complémentaires :

— **Gestionnaire produit** : valide, corrige ou rejette les attributs générés ;

— **Administrateur Hybris** : supervise le mapping et l'import des données validées ;

— **Service IA multimodal** : propose les enrichissements avec score de confiance ;

— **Apache Solr** : indexe les nouveaux attributs pour améliorer la recherche ;

— **Client final** : bénéficie indirectement d'une navigation plus pertinente.

1.6 Périmètre et contraintes

1.6.1 Périmètre fonctionnel

Le périmètre du PFE couvre les briques suivantes :

— enrichissement automatique d'attributs prioritaires ;

— validation humaine via interface dédiée ;

— intégration Hybris des attributs validés ;

— indexation Solr des nouveaux champs ;

— exposition des données enrichies via API.

1.6.2 Contraintes de réalisation

Les contraintes majeures identifiées sont :

— compatibilité avec le modèle de données Hybris existant ;

— maîtrise de la latence et du coût d'inférence LLM en traitement batch ;

— robustesse des échanges API (gestion d'erreurs, reprise, traçabilité) ;

— qualité éditoriale multilingue et cohérence des attributs générés.

1.7 Démarche méthodologique et planification

Le projet suit une démarche progressive structurée en phases :

— cadrage et étude de l'existant ;

— spécifications fonctionnelles et techniques ;

— conception d'architecture ;

— implémentation incrémentale par sprints ;

— tests, mesure des indicateurs et consolidation documentaire.

Cette organisation permet d'aligner la livraison technique avec les objectifs métier, tout en conservant une capacité d'ajustement continue.

1.8 Conclusion

Ce chapitre a posé les fondations du projet : contexte entreprise, problématique, objectifs, acteurs, périmètre et contraintes. Il constitue le référentiel de cadrage sur lequel s'appuient les choix d'analyse et de conception présentés dans la suite du rapport.

Chapitre

2

Analyse des besoins et étude de l'existant

2.1 Introduction

Dans la continuité du cadre général présenté au chapitre 1, ce chapitre précise les besoins à satisfaire et positionne la solution proposée par rapport à l'existant. L'objectif est double : formaliser des exigences exploitables pour la conception, puis justifier les choix techniques au regard des limites observées dans les approches actuelles.

2.2 Analyse des besoins

2.2.1 Besoins fonctionnels

Les besoins fonctionnels retenus couvrent l'ensemble du cycle d'enrichissement des données produit :

- Déclencher l'enrichissement automatique d'une fiche produit à partir d'images, d'attributs existants et de descriptions disponibles ;
- Générer un ensemble d'attributs structurés (couleur dominante et secondaire, matière, type/catégorie, motif, saisonnalité, description structurée) ;
- Affecter un score de confiance par attribut afin d'aider la décision métier ;
- Permettre la validation, la correction ou le rejet des attributs proposés via une interface de pilotage ;
- Tracer les décisions de validation (audit trail : utilisateur, date, valeur initiale, valeur finale) ;
- Injecter les attributs validés dans SAP Hybris avec gestion des statuts et mécanismes de reprise en cas d'erreur ;
- Réindexer les données enrichies dans Apache Solr pour améliorer filtres, facettes et pertinence de recherche ;
- Exposer les données enrichies via API REST pour la réutilisation par d'autres composants.

2.2.2 Besoins non fonctionnels

Au-delà des fonctionnalités, plusieurs exigences de qualité structurent la solution :

— **Performance** : traitement en mode batch compatible avec le volume catalogue (jusqu'à plusieurs milliers de produits par lot) ;

— **Fiabilité** : gestion robuste des erreurs d'API (retry, journalisation, reprise) ;

— **Auditabilité** : historisation des modifications pour garantir la traçabilité métier ;

— **Maintenabilité** : ajout de nouveaux attributs sans refonte majeure du pipeline ;

— **Interopérabilité** : alignement avec l'écosystème Java/Spring, Hybris et Solr ;

— **Évolutivité** : capacité à intégrer ultérieurement des fonctionnalités multilingues et des contrôles de non-conformité.

2.2.3 Acteurs et cas d'usage

La Figure 2.1 synthétise les interactions majeures entre les acteurs métier et la plateforme. Même si le schéma est générique, il met en évidence une idée centrale : chaque action sensible est conditionnée par l'authentification et s'inscrit dans une logique de gouvernance des données.

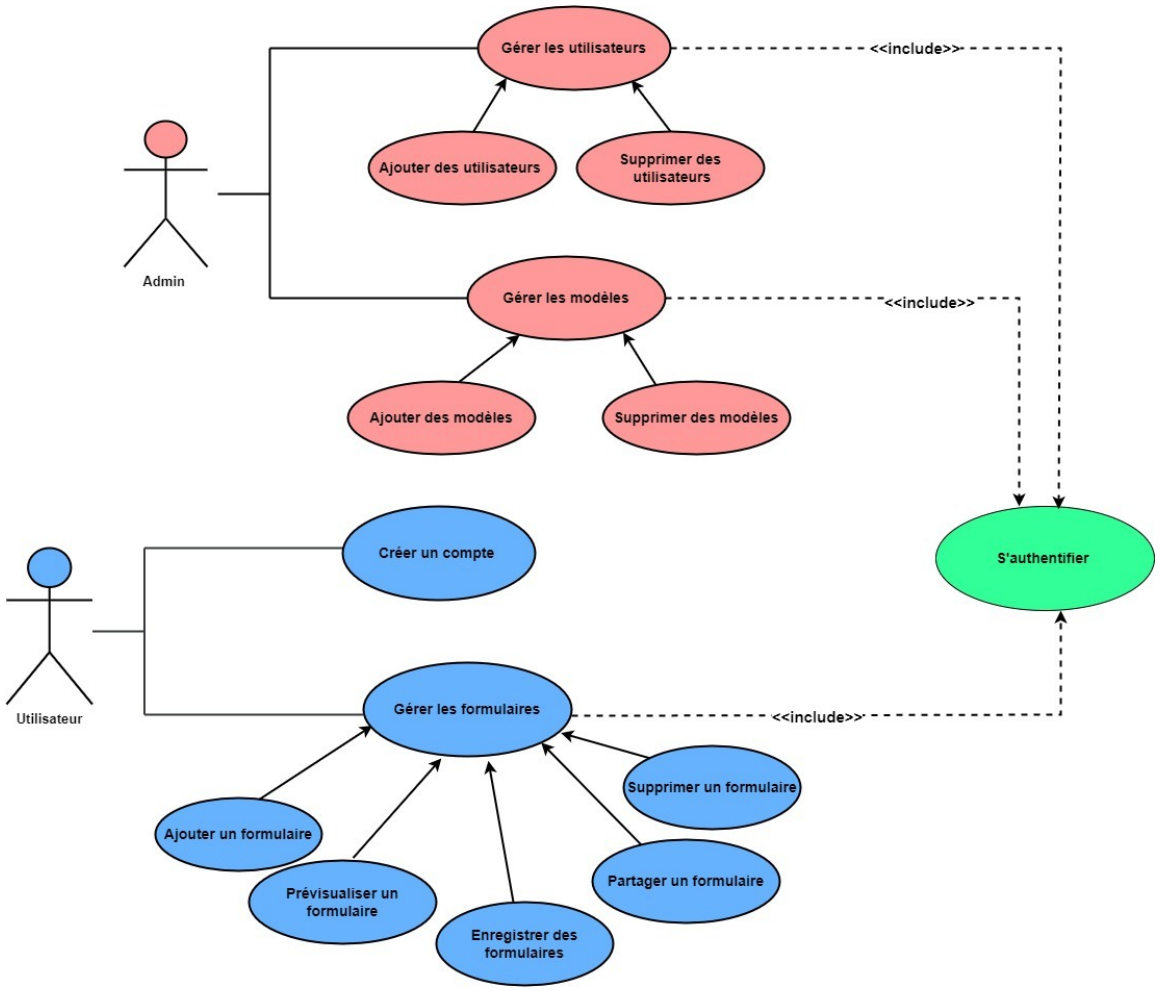

FIGURE 2.1 – Diagramme de cas d'usage global de la plateforme

En transposant ce principe au périmètre du PFE, les cas d'usage prioritaires sont :

— UC1 : Déclencher l'enrichissement d'un produit ;

— UC2 : Valider / corriger / rejeter les attributs générés ;

— UC3 : Importer les attributs validés dans Hybris ;

— UC4 : Réindexer les produits concernés dans Solr ;

— UC5 : Suivre les indicateurs de qualité des enrichissements.

2.2.4 Priorisation des user stories

La planification fonctionnelle initiale est résumée dans la Figure 2.2. Elle met en évidence un découpage progressif : d'abord les fonctions essentielles, puis les fonctions d'administration et de pilotage.

Sprint	User Story
Release 1	
Sprint 1:Impleméntation des fonctions essentielles	
S'inscrire	En tant qu'utilisateur, je peux m'enregistrer sur la plateforme en complétant un formulaire d'inscription. Ce formulaire vous demandera de renseigner votre adresse e-mail et de choisir un mot de passe.
S'authentifier	En tant qu'administrateur ou Utilisateur, je peux m'authentifier afin d' accéder à mon espace.
Gestion des formulaires	En tant qu'utilisateur, je peux créer un formulaire en ajoutant les champs nécessaires. Je peux personnaliser le formulaire en ajoutant des champs differents. je peux enrigistrer, prévisualiser,partager ou supprimer un formulaire.
Release 2	
Sprint 2: Impleméntation des fonctions de gestion	
Gestion des utilisateurs	En tant qu'administrateur je peux ajouter ou supprimer des utilisateurs
Gestion des modéles	En tant qu'administrateur je peux ajouter ou supprimer des templates.

FIGURE 2.2 – Priorisation initiale des user stories par release

2.3 Étude de l'existant

2.3.1 Objectif et protocole des tests

L'étude de l'existant ne se limite pas à une comparaison théorique. Elle s'appuie sur des essais pratiques réalisés sur des cas réels de fiches produit (texte, métadonnées, images multiples), afin de répondre à trois questions :

— les solutions **gratuites locales** sont-elles exploitables dans nos contraintes matérielles ;

— les **APIs gratuites** sont-elles suffisantes pour un flux e-commerce réel ;

— les **APIs payantes** apportent-elles le niveau de robustesse attendu pour industrialiser la solution.

Les tests ont été menés sur des scénarios de difficulté croissante : texte seul, image unique, multi-images, puis images + métadonnées.

2.3.2 Axe 1 – Solutions gratuites locales (Ollama)

2.3.2.1 Résultats observés

Les essais locaux avec des modèles multimodaux open-source (Gemma, Qwen-VL, Moondream) montrent une limite structurelle de l'infrastructure disponible :

— latence très élevée (plusieurs minutes même pour une réponse simple) ;

— forte pression CPU/RAM en l'absence de GPU dédié ;

— instabilités ou échecs sur les cas multi-images.

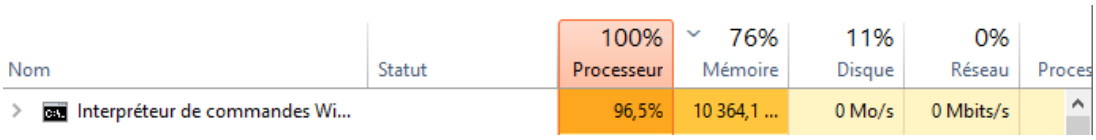

FIGURE 2.3 – Exemple d'échec et de surcharge en exécution locale

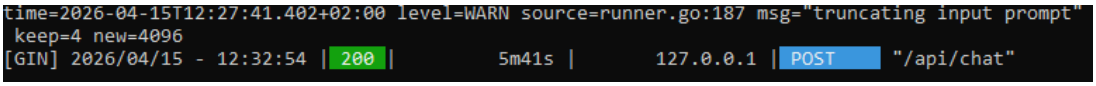

FIGURE 2.4 – Saturation des ressources machine en test local

2.3.2.2 Variante Colab GPU gratuit

Pour contourner la limite matérielle locale, des tests ont été réalisés sur Google Colab (GPU gratuit). Les modèles lourds restent néanmoins difficiles à exploiter opérationnellement : temps de réponse long, consommation GPU proche de la limite mémoire, et faible marge de stabilité pour des traitements en lot.

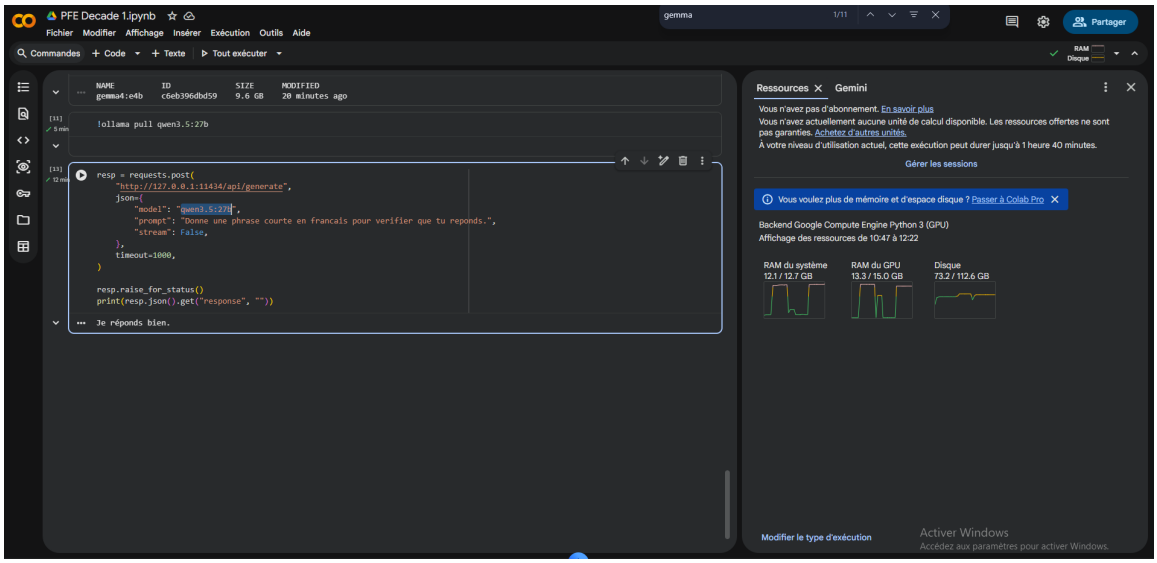

FIGURE 2.5 – Test Qwen 3.5 27B sur Colab GPU gratuit

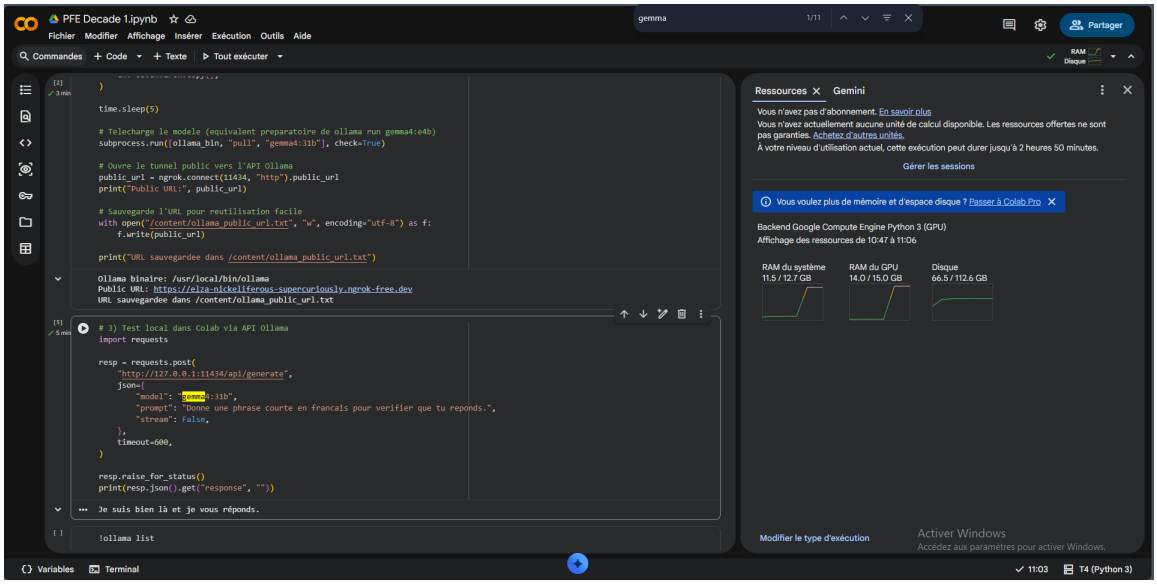

FIGURE 2.6 – Test Gemma 31B sur Colab GPU gratuit

Verdict de l'axe 1 : viable pour exploration et POC technique, non viable pour un pipeline de production stable dans notre contexte.

2.3.3 Axe 2 – APIs gratuites (OpenRouter, Groq, AI Studio Free)

2.3.3.1 OpenRouter (free tier)

Les tests OpenRouter confirment un comportement acceptable sur des entrées simples (texte seul, parfois image unique), mais les cas cibles du projet (multi-images + métadonnées) provoquent régulièrement :

— **context overflow** ;

— **HTTP 429** (Too Many Requests) ;

— dégradation de robustesse entre appels successifs.


```
You, last week | 1 author (You)
[
  {
    "productId": "2045144_895",
    "productVersion": 95,
    "imagesUsed": [
      "C:\\Users\\NejmeddinBENMAATOUG\\Desktop\\PFE\\working on models to test\\resources\\images\\2045144_895\\1.jpg",
      "C:\\Users\\NejmeddinBENMAATOUG\\Desktop\\PFE\\working on models to test\\resources\\images\\2045144_895\\2.jpg",
      "C:\\Users\\NejmeddinBENMAATOUG\\Desktop\\PFE\\working on models to test\\resources\\images\\2045144_895\\3.jpg",
      "C:\\Users\\NejmeddinBENMAATOUG\\Desktop\\PFE\\working on models to test\\resources\\images\\2045144_895\\4.jpg"
    ],
    "error": "429 Client Error: Too Many Requests for url: https://openrouter.ai/api/v1/chat/completions"
  }
]      You, last week • openRouter free multimodal models : I test thei…
```


FIGURE 2.7 – Exemple de limite contextuelle sur OpenRouter en multi-images

2.3.3.2 Groq (free tier)

Sur Groq, la latence est bonne, mais la contrainte TPM (tokens par minute) devient bloquante sur un produit enrichi avec images et métadonnées. Les erreurs `HTTP 413` et `rate_limit_exceeded` empêchent la continuité du flux.


```
"groqAlignment": [
  {
    "productId": "2043311_136",
    "category": "clothing",
    "error": "Groq API error 413: {\"error\":{\"message\":\"Request too large for model `llama-3.3-70b-versatile` in organization `org_01k8d6bm9zegab2xs3bnr8gjcq` servi
    Limit 12000, Requested 46471, please reduce your message size and try again. Need more tokens? Upgrade to Dev Tier today at https://console.groq.com/settings/billin
  }
]
```


FIGURE 2.8 – Erreur de quota tokens observée sur Groq

2.3.3.3 Google AI Studio (gratuit)

Les essais montrent une meilleure qualité de réponse que plusieurs alternatives gratuites, mais les mêmes limites restent présentes : quotas gratuits serrés, erreurs 429 sous charge, et absence de garantie de service.


```
    },
    "error": "Ollama API error 500: {\"error\":\"Failed to create new sequence: SameBatch may not be specified within numKeep (index: 3 numKeep: 4 SameBatch: 1937)\"}",
    "explication":"Modèle trop lourd/instable pour la session GPU actuelle (ici qwen3.5:27b est très exigeant). Ollama retourne une erreur 500 générique, sans enrichissement ef
  }
]
```


FIGURE 2.9 – Exemple d'utilisation AI Studio avec contrainte de ressources gratuites

Verdict de l'axe 2 : utile pour prototypage rapide, insuffisant pour une chaîne fiable d'enrichissement e-commerce.

2.3.4 Axe 3 – APIs payantes (candidats de production)

Les résultats des axes 1 et 2 conduisent à considérer des APIs payantes pour la phase d'industrialisation. Le Tableau 2.1 synthétise les options les plus crédibles.

TABLE 2.1 – Comparaison des APIs payantes candidates pour la production

Solution	Forces	Latence	Coût	Risque principal
GPT-4o (OpenAI)	Très bonne qualité multimodale, sortie JSON robuste	Faible à moyenne	Élevé	Coût à l'échelle et gouvernance des données
Gemini 2.5 (Google)	Bonne qualité visuelle, contexte large	Moyenne	Moyen à élevé	Stabilité des quotas selon plan choisi
Claude Sonnet (Anthropic)	Excellente compréhension sémantique, bonne rigueur de sortie	Faible à moyenne	Élevé	Coût et intégration selon stack retenue

Sur le plan métier, ces offres payantes répondent mieux aux exigences critiques du projet : robustesse, continuité de service, meilleur support des scénarios multi-images et gestion contrôlée des quotas.

2.3.5 Synthèse de décision

L'étude expérimentale met en évidence une trajectoire claire :

— **Gratuit local** : adapté à la recherche et aux démonstrations ponctuelles, mais trop instable pour un flux opérationnel ;

— **APIs gratuites** : utiles pour explorer rapidement des modèles, mais contraintes de quotas incompatibles avec un usage soutenu ;

— **APIs payantes** : option la plus réaliste pour atteindre la robustesse attendue en environnement e-commerce.

La stratégie recommandée est donc hybride : utiliser les offres gratuites pour le prototypage initial, puis basculer vers une API payante pour la phase de validation industrielle et de déploiement progressif.

2.4 Conclusion

Ce chapitre a transformé le besoin initial en exigences structurées et mesurables, puis a démontré les limites de l'existant et l'intérêt d'une approche hybride orientée IA multimodale. Les éléments établis ici serviront de base à la spécification détaillée et à la conception d'architecture du chapitre suivant.